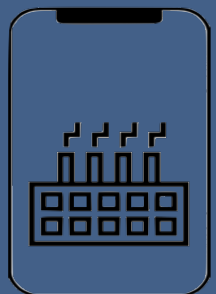


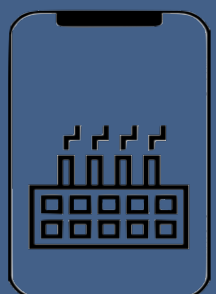
Big O



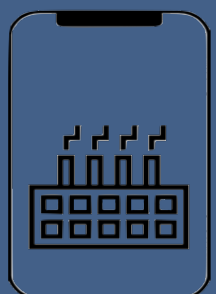
Big O

Objectives:

- What is Recursion?
- Why do we need Recursion?
- The Logic behind the Recursion
- Recursive vs Iterative Solutions
- How to Write Recursion in 3 Steps?
- Find Fibonacci Series using Recursion

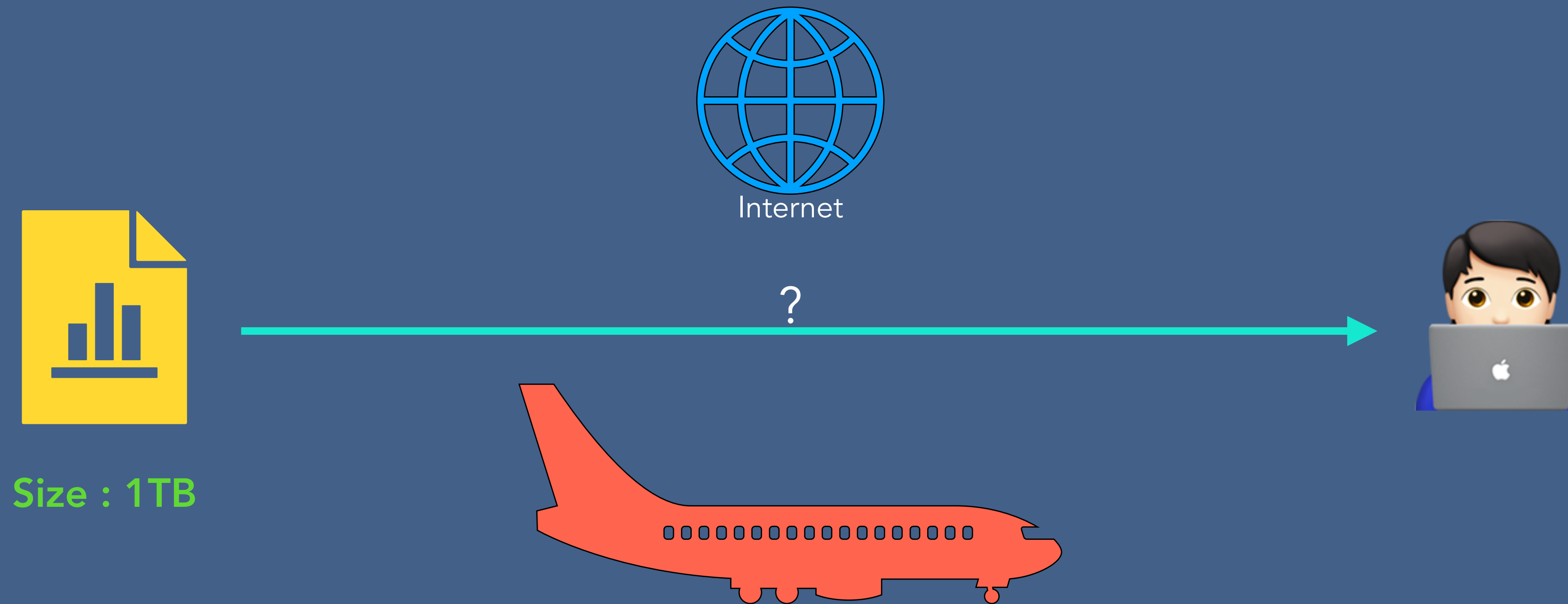


What is Big O?

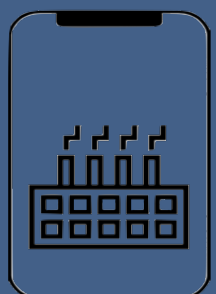


What is Big O?

Big O is the language and metric we use to describe the efficiency of algorithms.

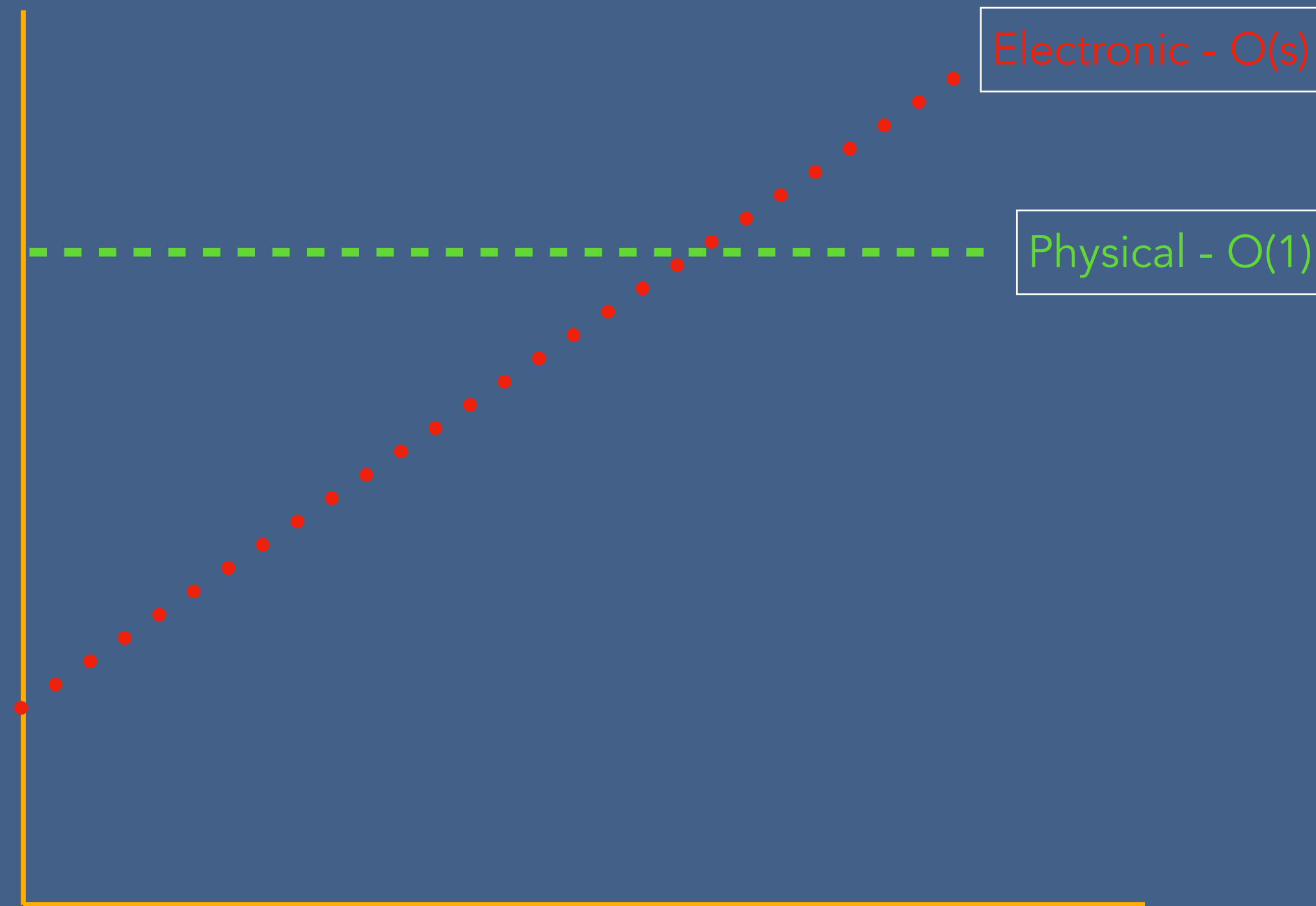


Time Complexity : A way of showing how the runtime of a function increases as the size of input increases.

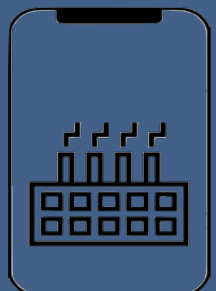


What is Big O?

Big O is the language and metric we use to describe the efficiency of algorithms.



Time Complexity : A way of showing how the runtime of a function increases as the size of input increases.



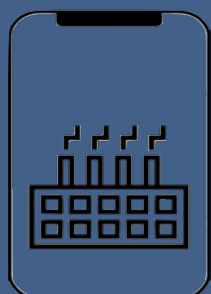
What is Big O?

Types of Runtimes:

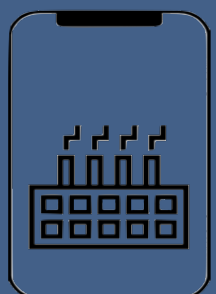
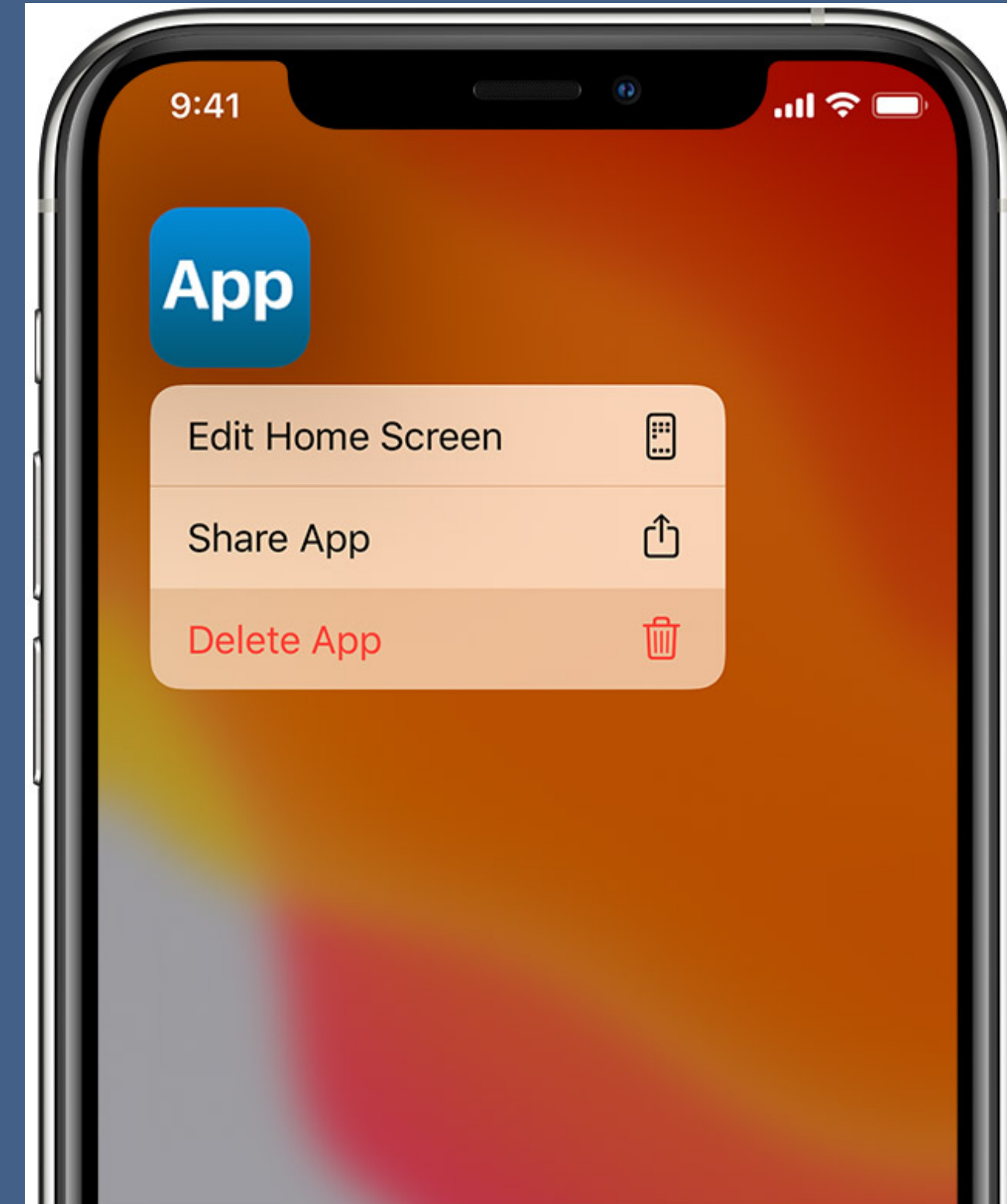
$O(N)$, $O(N^2)$, $O(2^N)$



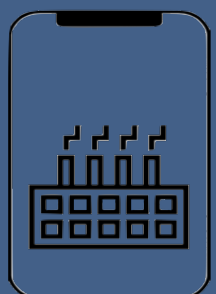
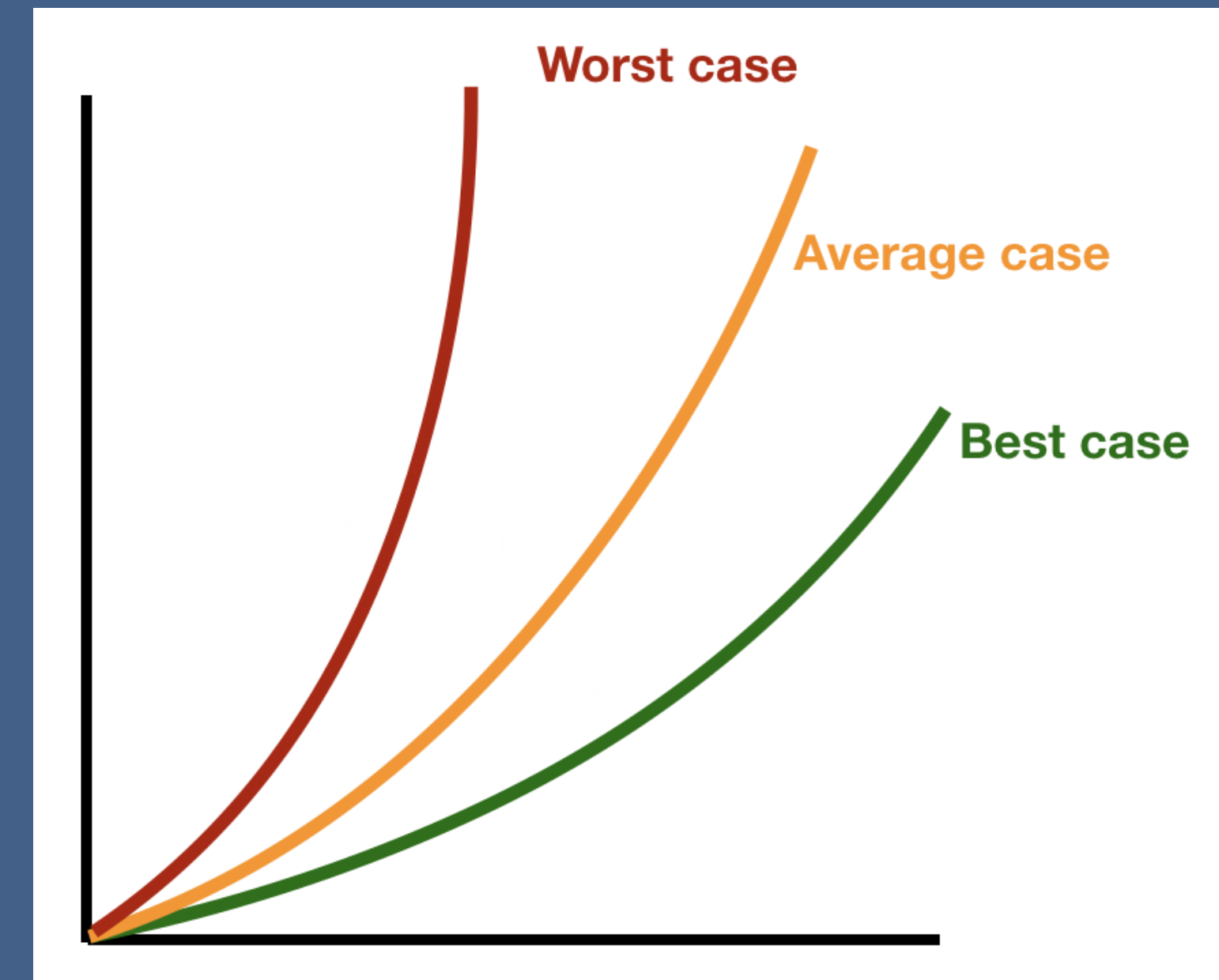
Time complexity : $O(wh)$



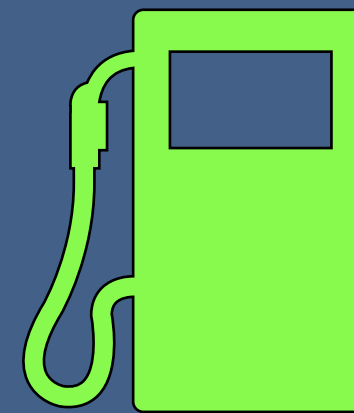
What is Big O?



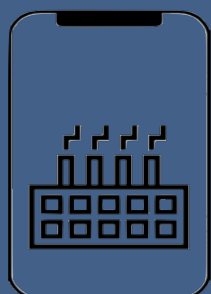
Big O Notations



Big O Notations

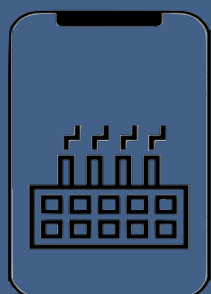
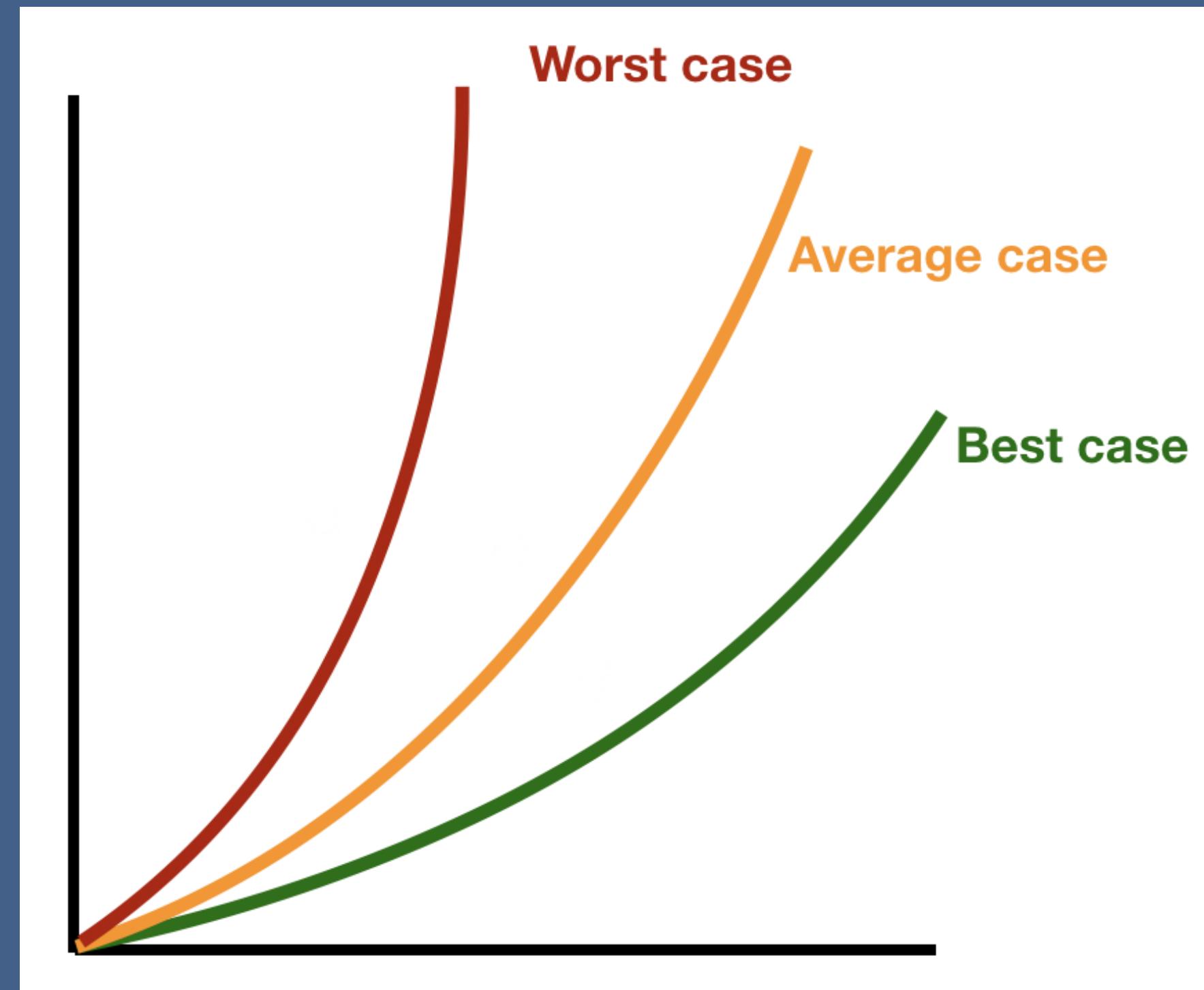


- City traffic - 20 liters
- Highway - 10 liters
- Mixed condition - 15 liters



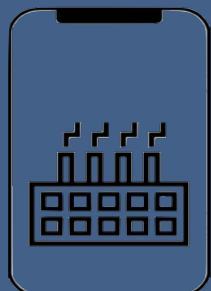
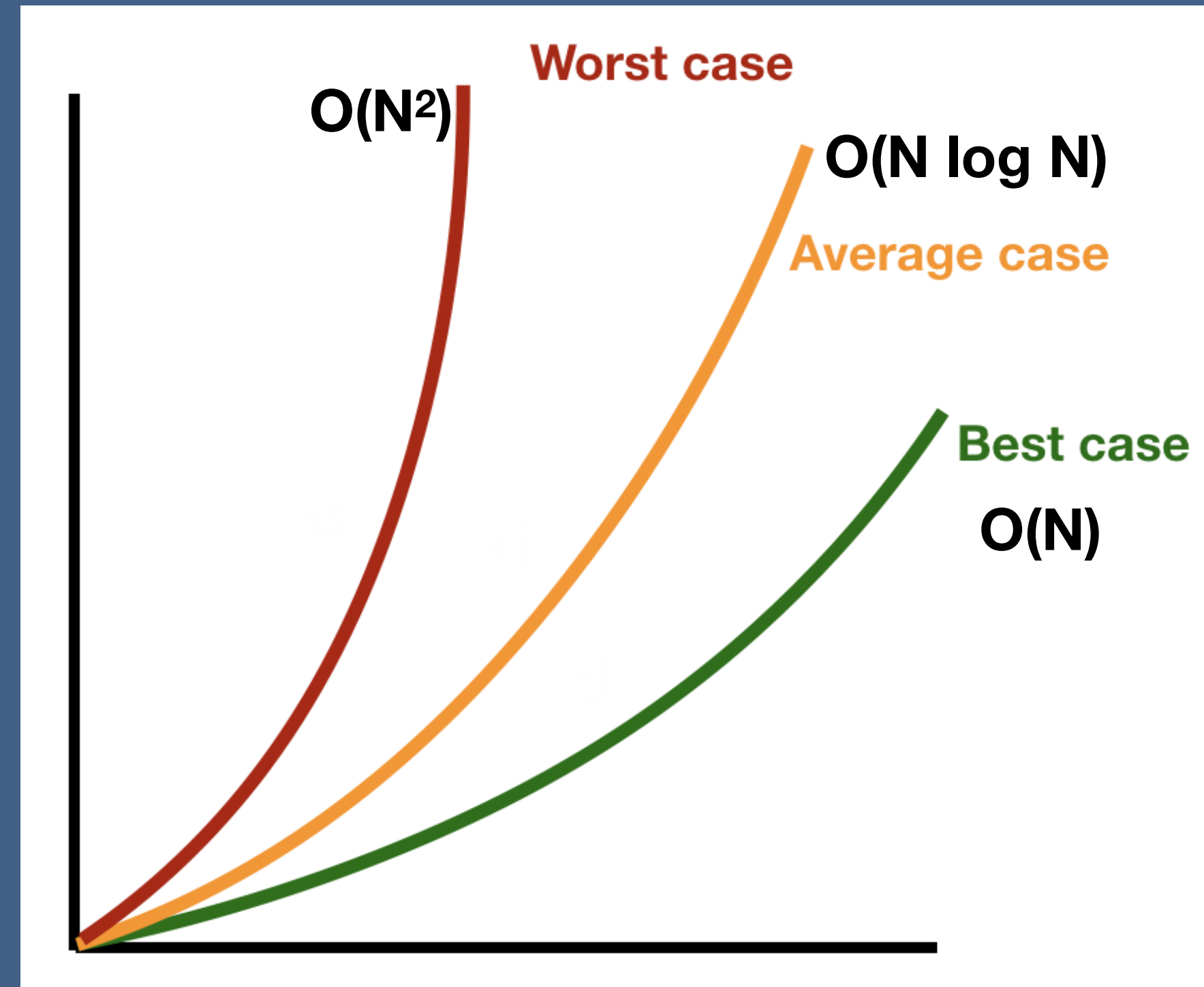
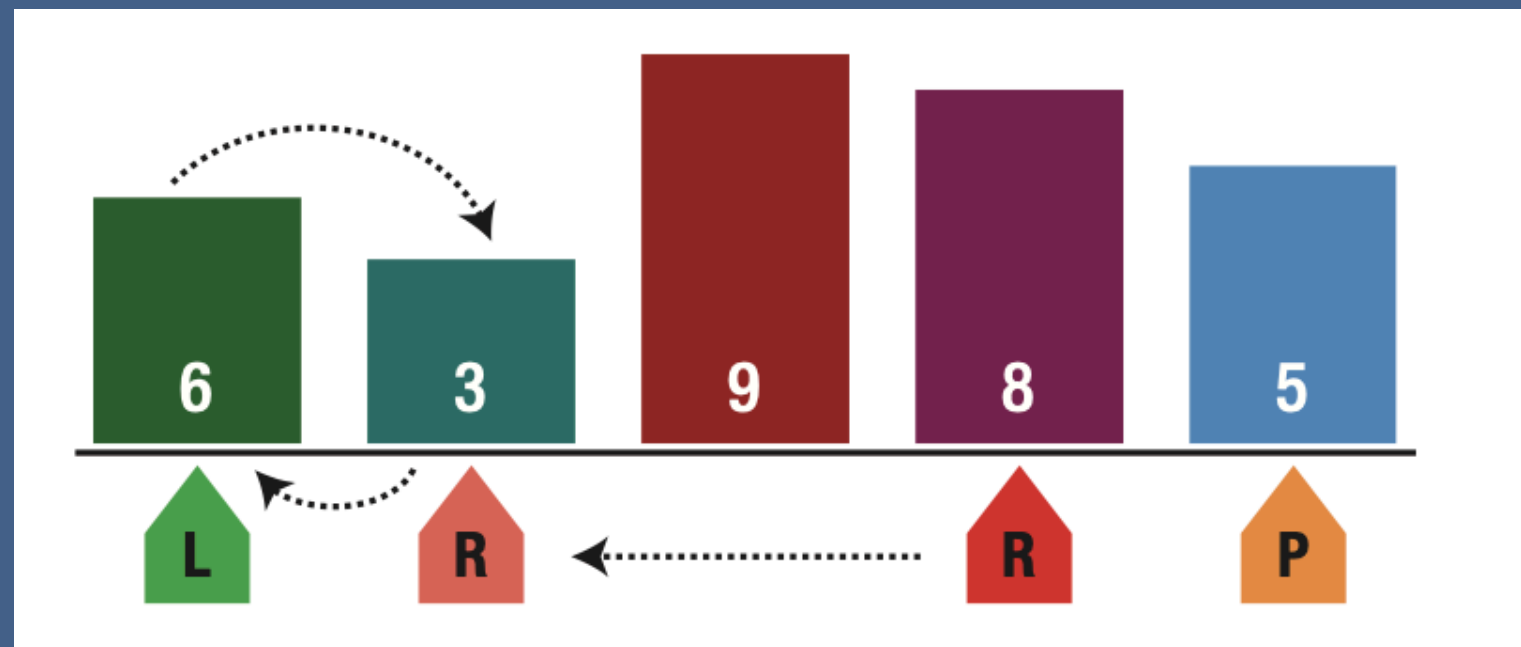
Big O Notations

- Best case
- Average case
- Worst case



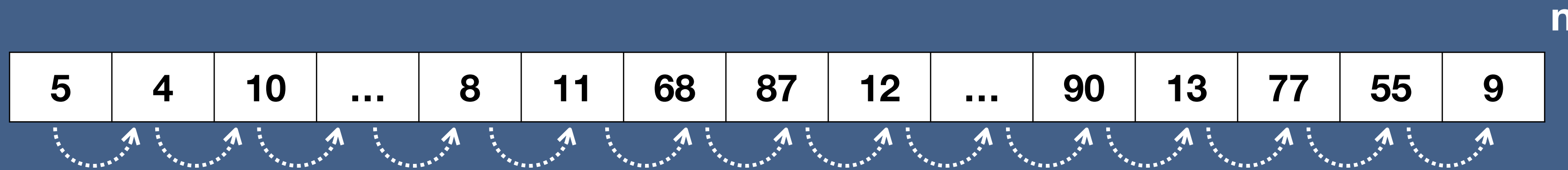
Big O Notations

Quick sort algorithm



Big O Notations

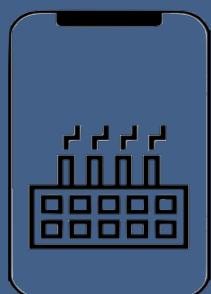
- **Big O** : It is a complexity that is going to be less or equal to the worst case.
- **Big - Ω (Big-Omega)** : It is a complexity that is going to be at least more than the best case.
- **Big Theta (Big - Θ)** : It is a complexity that is within bounds of the worst and the best cases.



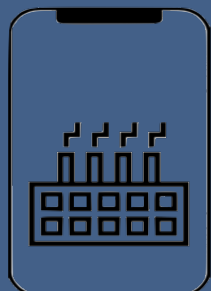
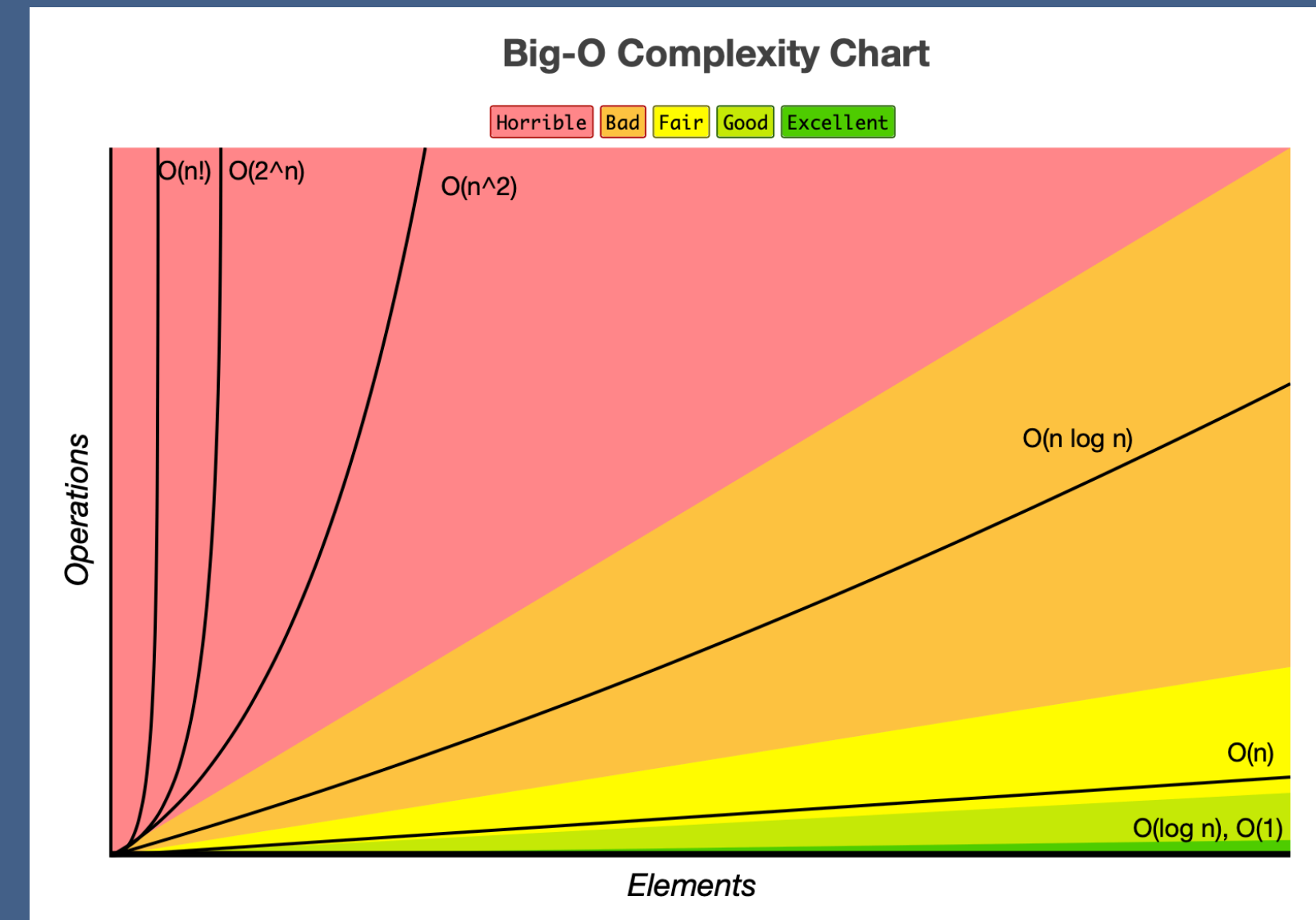
Big O - $O(N)$

Big Ω - $\Omega(1)$

Big Θ - $\Theta(n/2)$



Runtime Complexities

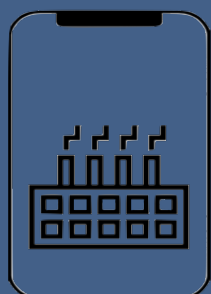


Runtime Complexities

Complexity	Name	Sample
$O(1)$	Constant	Accessing a specific element in array
$O(N)$	Linear	Loop through array elements
$O(\log N)$	Logarithmic	Find an element in sorted array
$O(N^2)$	Quadratic	Looking at every index in the array twice
$O(2^N)$	Exponential	Double recursion in Fibonacci

$O(1)$ - Constant time

```
int[] array = {1, 2, 3, 4, 5}
array[0] // It takes constant time to access first element
```



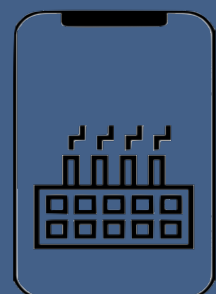
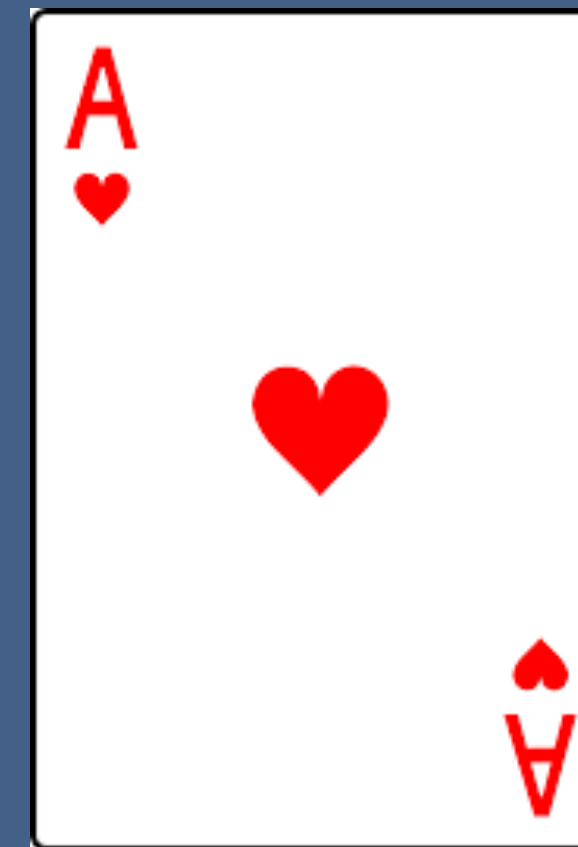
Runtime Complexities

Complexity	Name	Sample
$O(1)$	Constant	Accessing a specific element in array
$O(N)$	Linear	Loop through array elements
$O(\log N)$	Logarithmic	Find an element in sorted array
$O(N^2)$	Quadratic	Looking at every index in the array twice
$O(2^N)$	Exponential	Double recursion in Fibonacci

$O(1)$ - Constant time



random card

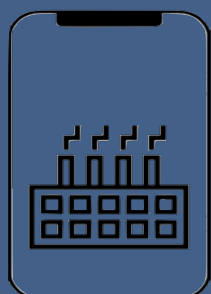


Runtime Complexities

Complexity	Name	Sample
$O(1)$	Constant	Accessing a specific element in array
$O(N)$	Linear	Loop through array elements
$O(\log N)$	Logarithmic	Find an element in sorted array
$O(N^2)$	Quadratic	Looking at every index in the array twice
$O(2^N)$	Exponential	Double recursion in Fibonacci

$O(N)$ - Linear time

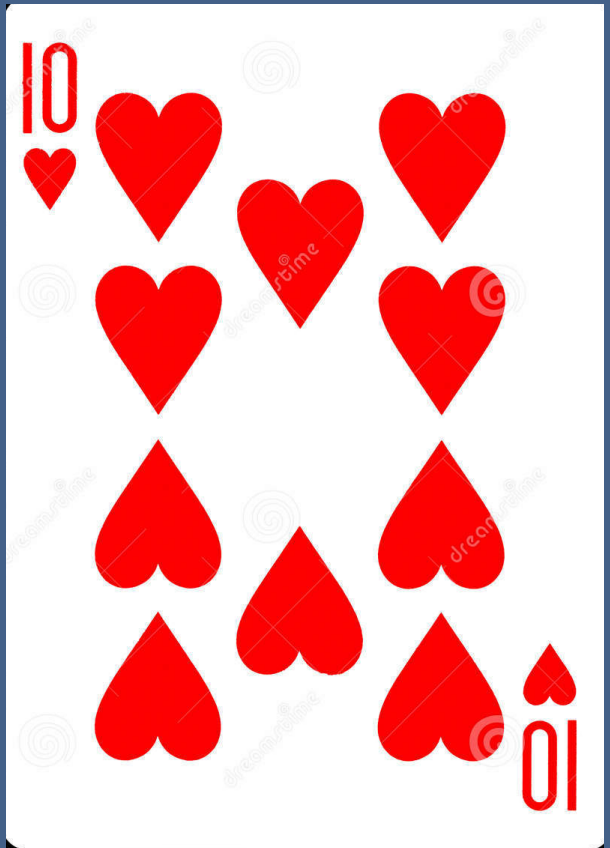
```
int[] custArray = {1, 2, 3, 4, 5}
for (int i = 0; i < custArray.length; i++) {
    System.out.println(custArray[i]);
}
//linear time since it is visiting every element of array
```



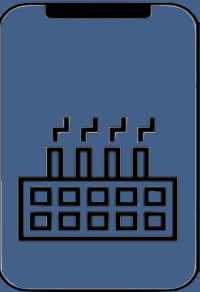
Runtime Complexities

Complexity	Name	Sample
$O(1)$	Constant	Accessing a specific element in array
$O(N)$	Linear	Loop through array elements
$O(\log N)$	Logarithmic	Find an element in sorted array
$O(N^2)$	Quadratic	Looking at every index in the array twice
$O(2^N)$	Exponential	Double recursion in Fibonacci

$O(N)$ - Linear time



Specific card

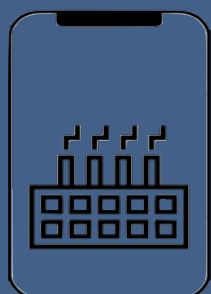


Runtime Complexities

Complexity	Name	Sample
$O(1)$	Constant	Accessing a specific element in array
$O(N)$	Linear	Loop through array elements
$O(\log N)$	Logarithmic	Find an element in sorted array
$O(N^2)$	Quadratic	Looking at every index in the array twice
$O(2^N)$	Exponential	Double recursion in Fibonacci

$O(\log N)$ - Logarithmic time

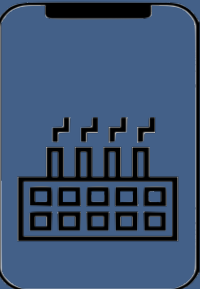
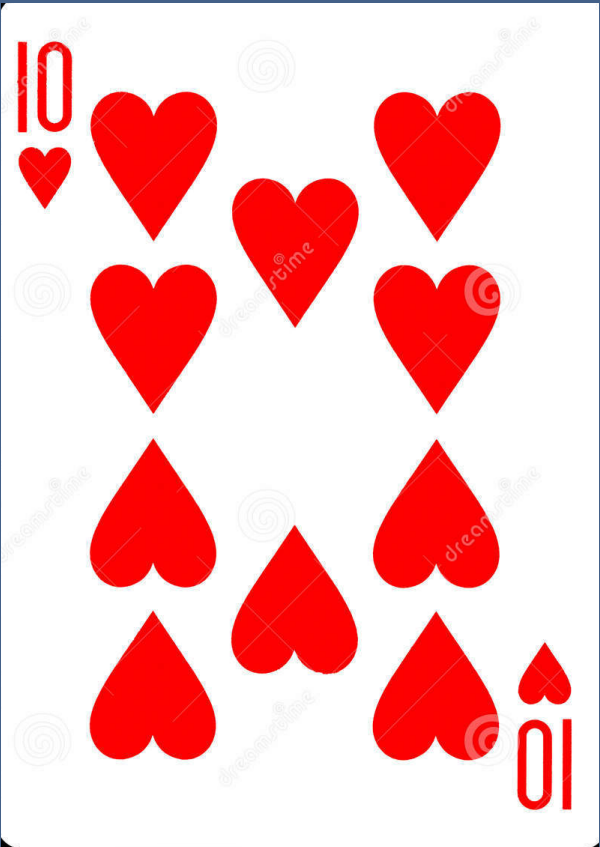
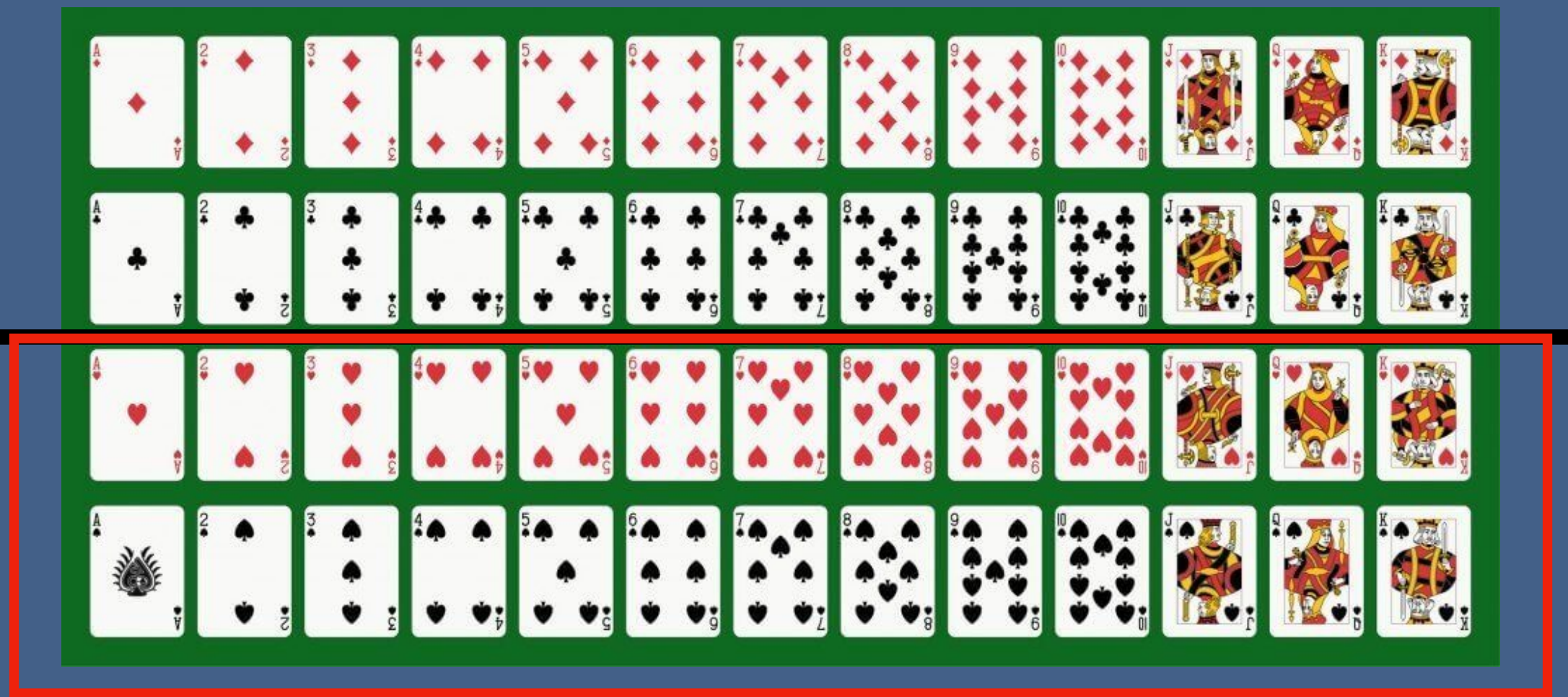
```
int[] custArray = {1, 2, 3, 4, 5}
for (int i = 0; i < custArray.length; i+=3) {
    System.out.println(custArray[i]);
}
//logarithmic time since it is visiting only some elements
```



Runtime Complexities

Complexity	Name	Sample
$O(1)$	Constant	Accessing a specific element in array
$O(N)$	Linear	Loop through array elements
$O(\log N)$	Logarithmic	Find an element in sorted array
$O(N^2)$	Quadratic	Looking at every index in the array twice
$O(2^N)$	Exponential	Double recursion in Fibonacci

$O(\log N)$ - Logarithmic time



Runtime Complexities

Complexity	Name	Sample
$O(1)$	Constant	Accessing a specific element in array
$O(N)$	Linear	Loop through array elements
$O(\log N)$	Logarithmic	Find an element in sorted array
$O(N^2)$	Quadratic	Looking at every index in the array twice
$O(2^N)$	Exponential	Double recursion in Fibonacci

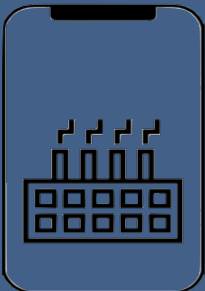
$O(\log N)$ - Logarithmic time

Binary search

```
search 9 within [1,5,8,9,11,13,15,19,21]
compare 9 to 11 → smaller
search 9 within [1,5,8,9]
compare 9 to 8 → bigger
search 9 within [9]
compare 9 to 9
return
```

```
N = 16
N = 8 /* divide by 2 */
N = 4 /* divide by 2 */
N = 2 /* divide by 2 */
N = 1 /* divide by 2 */
```

$$2^k = N \rightarrow \log_2 N = k$$

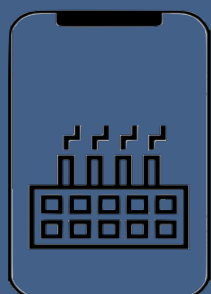


Runtime Complexities

Complexity	Name	Sample
$O(1)$	Constant	Accessing a specific element in array
$O(N)$	Linear	Loop through array elements
$O(\log N)$	Logarithmic	Find an element in sorted array
$O(N^2)$	Quadratic	Looking at every index in the array twice
$O(2^N)$	Exponential	Double recursion in Fibonacci

$O(N^2)$ - Quadratic time

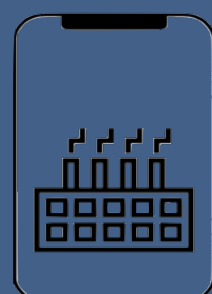
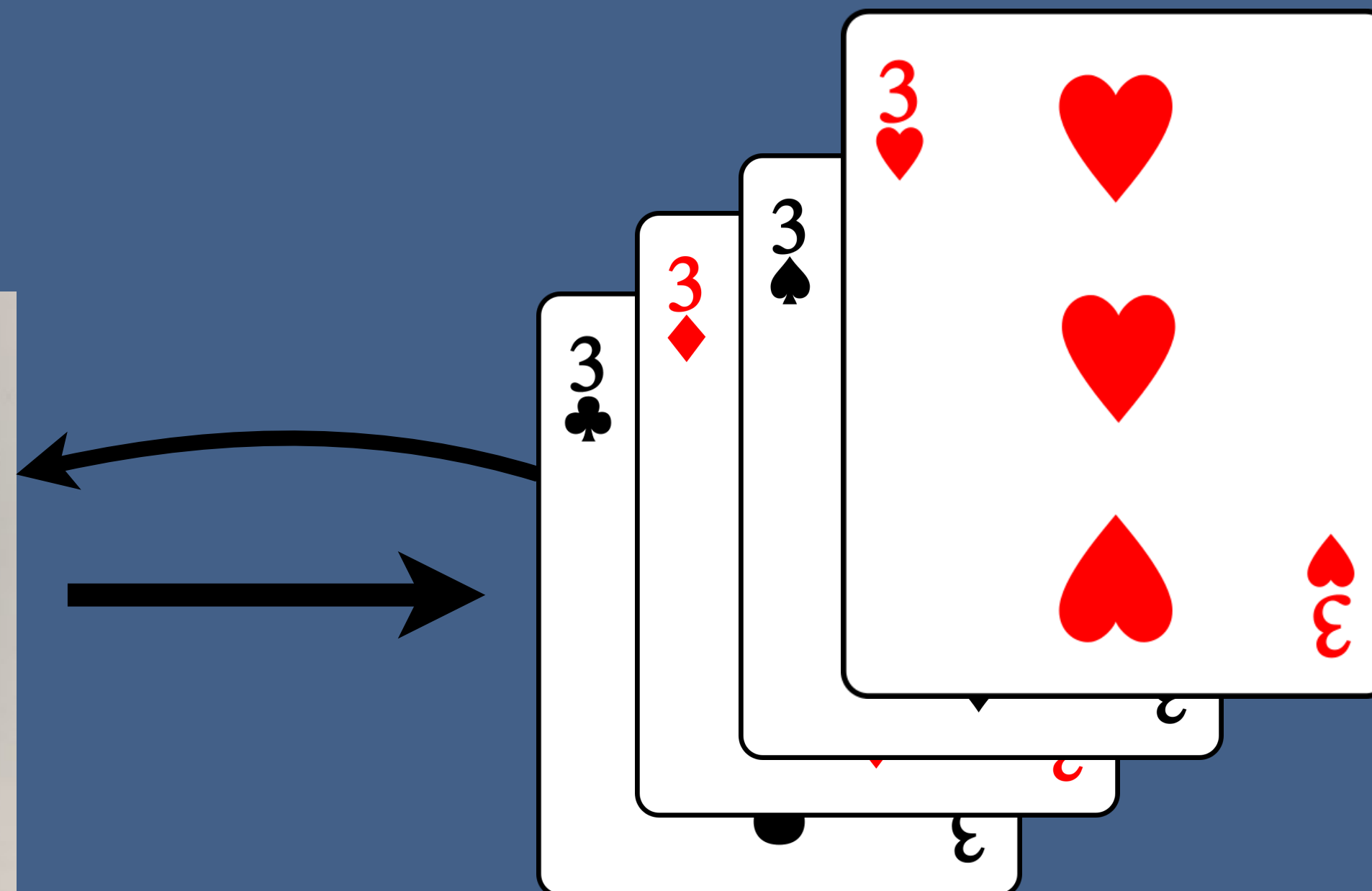
```
int[] custArray = {1, 2, 3, 4, 5}
for (int i = 0; i < custArray.length; i++) {
    for (int j = 0; j < custArray.length; j++) {
        System.out.println(custArray[i]);
    }
}
```



Runtime Complexities

Complexity	Name	Sample
$O(1)$	Constant	Accessing a specific element in array
$O(N)$	Linear	Loop through array elements
$O(\log N)$	Logarithmic	Find an element in sorted array
$O(N^2)$	Quadratic	Looking at every index in the array twice
$O(2^N)$	Exponential	Double recursion in Fibonacci

$O(N^2)$ - Quadratic time

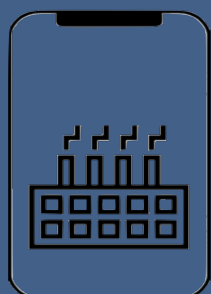


Runtime Complexities

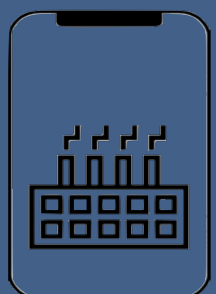
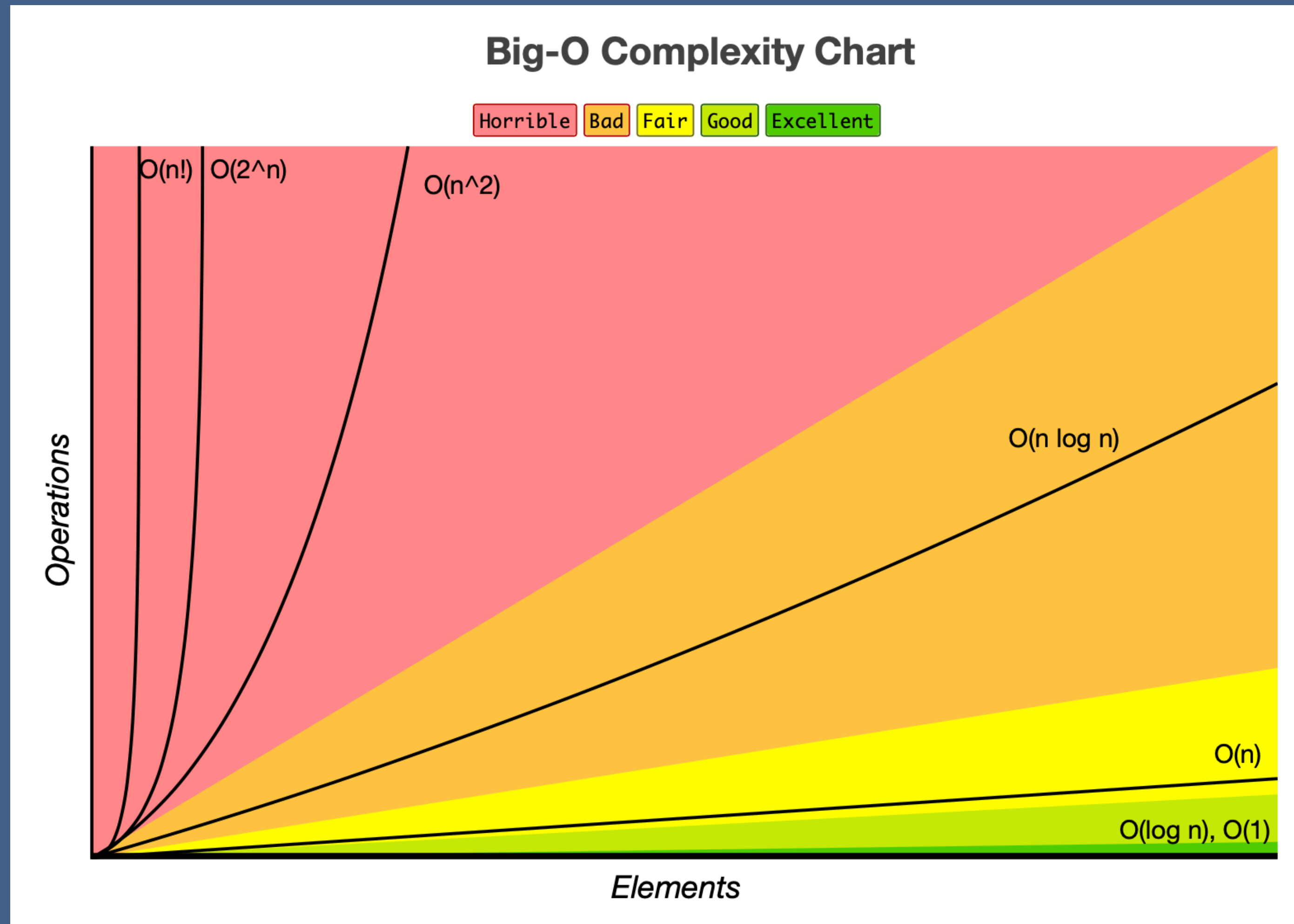
Complexity	Name	Sample
$O(1)$	Constant	Accessing a specific element in array
$O(N)$	Linear	Loop through array elements
$O(\log N)$	Logarithmic	Find an element in sorted array
$O(N^2)$	Quadratic	Looking at every index in the array twice
$O(2^N)$	Exponential	Double recursion in Fibonacci

$O(2^N)$ - Exponential time

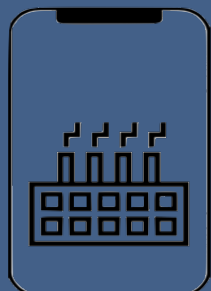
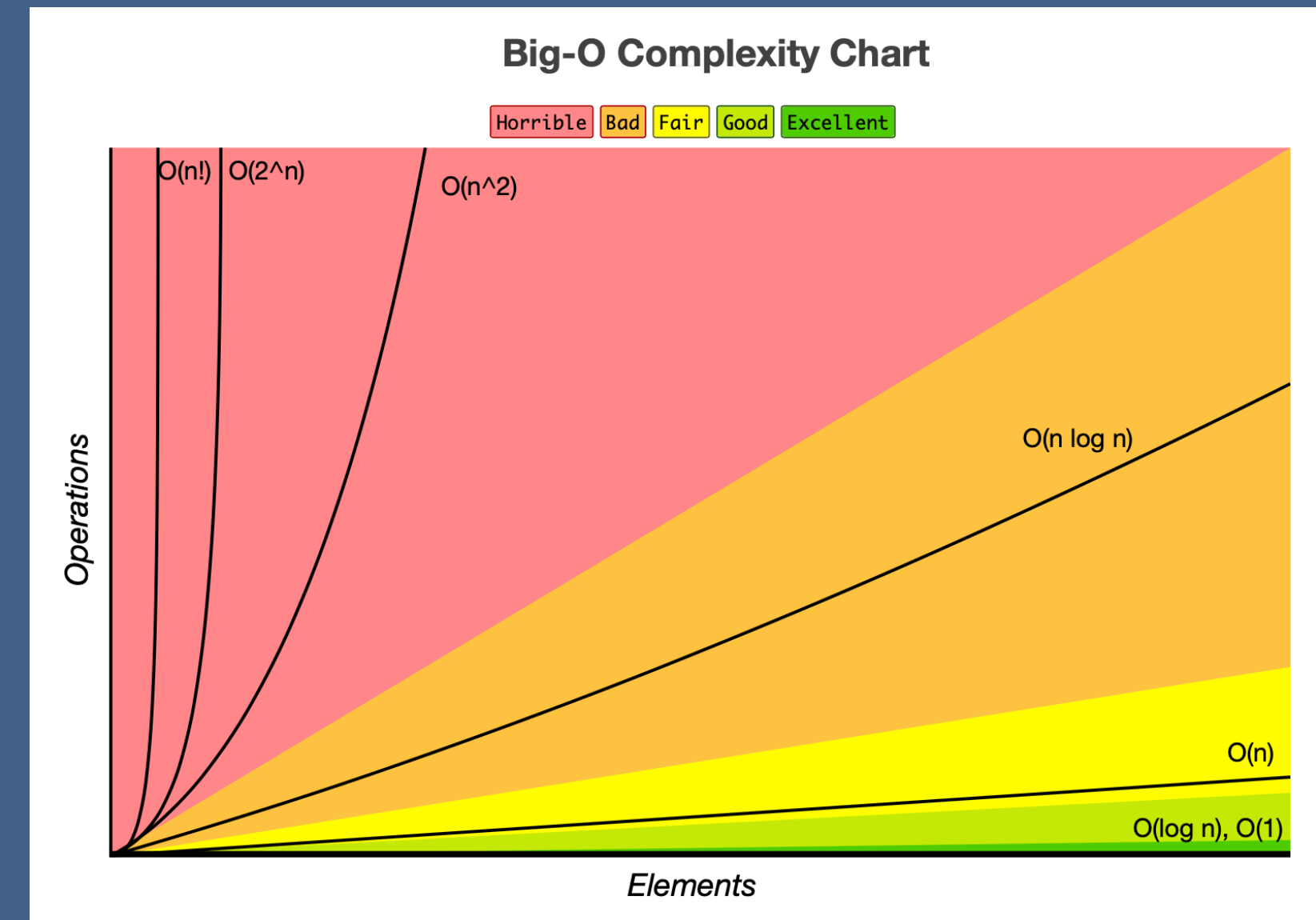
```
public int fibonacci(int n) {  
    if (n==0 || n==1) {  
        return n;  
    }  
    return fibonacci(n-1) + fibonacci(n-2);  
}
```



Runtime Complexities



Space Complexity



Space Complexity

an array of size n

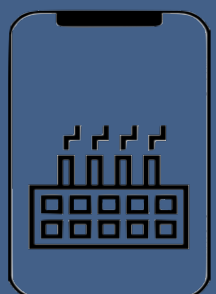
$$a = \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix}$$

$O(n)$

an array of size $n*n$

$$a = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix}$$

$O(n^2)$

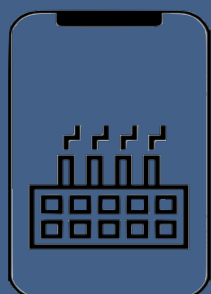


Space Complexity

```
static int sum(int n) {  
    if (n <= 0) {  
        return 0;  
    }  
    return n + sum(n-1);  
}
```

```
1  sum(3)  
2  → sum(2)  
3  → sum(1)  
4  → sum(0)
```

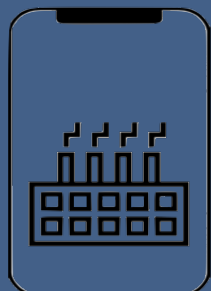
Space complexity : $O(n)$



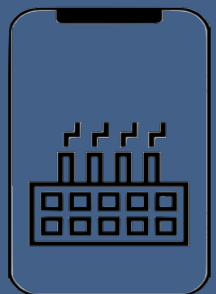
Space Complexity

```
static int pairSumSequence(int n) {  
    var sum = 0;  
    for (int i = 0; i <= n; i++) {  
        sum = sum + pairSum(i, i+1);  
    }  
    return sum;  
}  
  
static int pairSum(int a, int b) {  
    return a + b;  
}
```

Space complexity : $O(1)$



Drop Constants and Non Dominant Terms



Drop Constants and Non Dominant Terms

Drop Constant

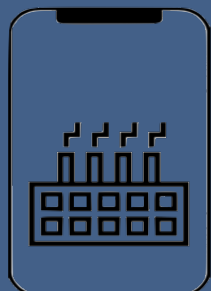
$$O(2N) \longrightarrow O(N)$$

Drop Non Dominant Terms

$$O(N^2+N) \longrightarrow O(N^2)$$

$$O(N+\log N) \longrightarrow O(N)$$

$$O(2 \cdot 2^N + 1000N^{100}) \longrightarrow O(2^N)$$

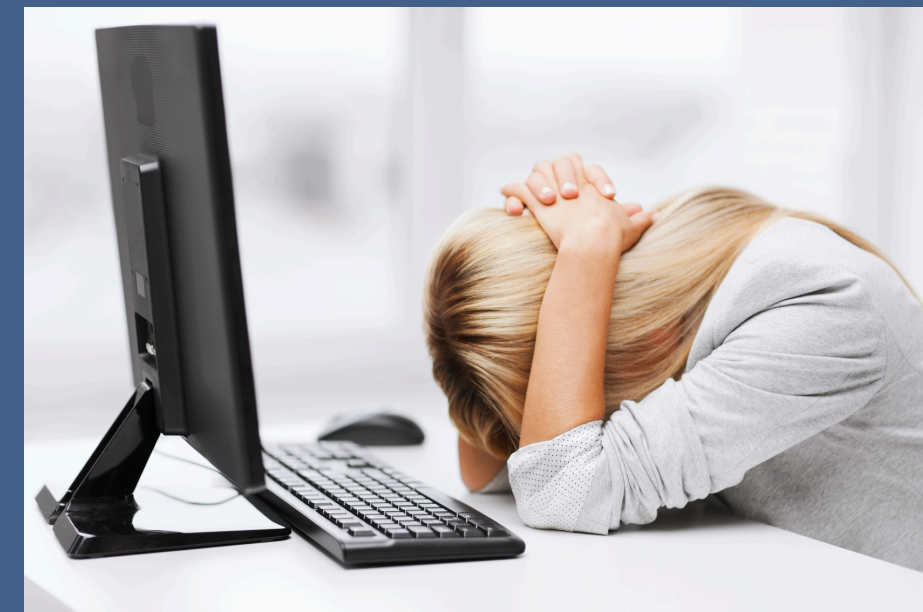


Drop Constants and Non Dominant Terms

- It is very possible that $O(N)$ code is faster than $O(1)$ code for specific inputs
- Different computers with different architectures have different constant factors.



Fast computer
Fast memory access
Lower constant

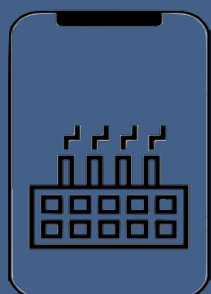


Slow computer
Slow memory access
Higher constant

- Different algorithms with the same basic idea and computational complexity might have slightly different constants

Example: $a*(b-c)$ **vs** $a*b - a*c$

- As $n \rightarrow \infty$, constant factors are not really a big deal



Add vs Multiply

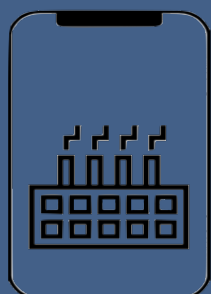
```
for (a=0; arrayA.length; a++) {  
    System.out.println(arrayA[a]);  
}  
  
for (b=0; arrayB.length; b++) {  
    System.out.println(arrayB[b]);  
}
```

Add the Runtimes: $O(A + B)$

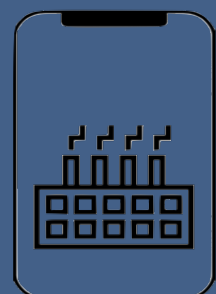
```
for (a=0; arrayA.length; a++) {  
    for (b=0; arrayB.length; b++) {  
        System.out.println(arrayB[b] + arrayA[a]);  
    }  
}
```

Multiply the Runtimes: $O(A * B)$

- If your algorithm is in the form "do this, then when you are all done, do that" then you add the runtimes.
- If your algorithm is in the form "do this for each time you do that" then you multiply the runtimes.



How to measure the codes using Big O?



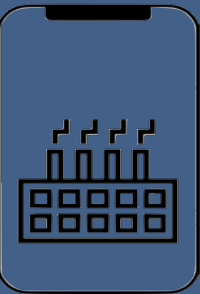
How to measure the codes using Big O?

No	Description	Complexity
Rule 1	Any assignment statements and if statements that are executed once regardless of the size of the problem	$O(1)$
Rule 2	A simple "for" loop from 0 to n (with no internal loops)	$O(n)$
Rule 3	A nested loop of the same type takes quadratic time complexity	$O(n^2)$
Rule 4	A loop, in which the controlling parameter is divided by two at each step	$O(\log n)$
Rule 5	When dealing with multiple statements, just add them up	

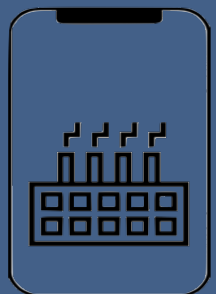
sampleArray **5 4 10 ... 8 11 68 87 ...**

```
Public static void findBiggestNumber([] sampleArray) {
    var biggestNumber = sampleArray[0]; .....→ O(1)
    for (index=1; sampleArray.length; index++) { .....→O(n) }
        if (sampleArray[index] > biggestNumber) {.....→ O(1) } .....→ O(n)
            biggestNumber = sampleArray[index]; .....→ O(1) } .....→ O(1)
        }
    }
    System.out.println(biggestNumber); .....→ O(1)
}
```

Time complexity : $O(1) + O(n) + O(1) = O(n)$



How to measure Recursive Algorithm?



How to measure Recursive Algorithm?

sampleArray

5	4	10	...	8	11	68	87	10
---	---	----	-----	---	----	----	----	----

```
public int findMaxNumRec(int [] sampleArray, int n):  
    if (n == 1) {  
        return sampleArray[0];  
    }  
    return max(sampleArray[n-1], findMaxNumRec(sampleArray, n-1));
```

Explanation:

A =

11	4	12	7
----	---	----	---

 n = 4

findMaxNumRec(A,4) $\longrightarrow$ max(A[4-1],12) $\longrightarrow$ max(7,12)=12



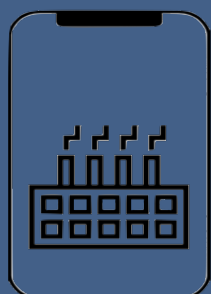
findMaxNumRec(A,3) $\longrightarrow$ max(A[3-1],11) $\longrightarrow$ max(12,11)=12



findMaxNumRec(A,2) $\longrightarrow$ max(A[2-1],11) $\longrightarrow$ max(4,11)=11



findMaxNumRec(A,1) $\longrightarrow$ A[0]=11



How to measure Recursive Algorithm?

sampleArray

5	4	10	...	8	11	68	87	10
---	---	----	-----	---	----	----	----	----

```
public int findMaxNumRec(int [] sampleArray, int n): .....→ M(n)
    if (n == 1) { .....→ O(1)
        return sampleArray[0]; .....→ O(1)
    }
    return max(sampleArray[n-1], findMaxNumRec(sampleArray, n-1)); .....→ M(n-1)
```

$$M(n) = O(1) + M(n-1)$$

$$M(1) = O(1)$$

$$M(n-1) = O(1) + M((n-1)-1)$$

$$M(n-2) = O(1) + M((n-2)-1)$$



$$M(n) = 1 + M(n-1)$$

$$= 1 + (1 + M((n-1)-1))$$

$$= 2 + M(n-2)$$

$$= 2 + 1 + M((n-2)-1)$$

$$= 3 + M(n-3)$$

.

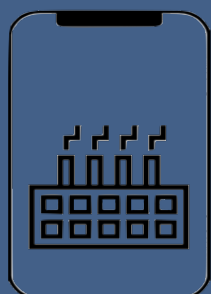
.

$$= a + M(n-a)$$

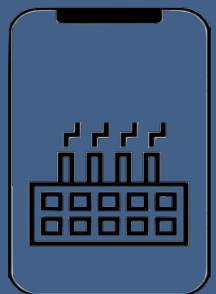
$$= n-1 + M(n-(n-1))$$

$$= n-1 + 1$$

$$= n$$



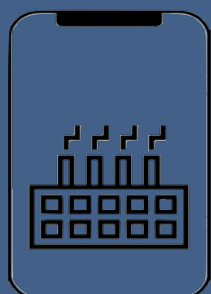
How to measure Recursive Algorithm with multiple calls?



How to measure Recursive Algorithm with multiple calls?

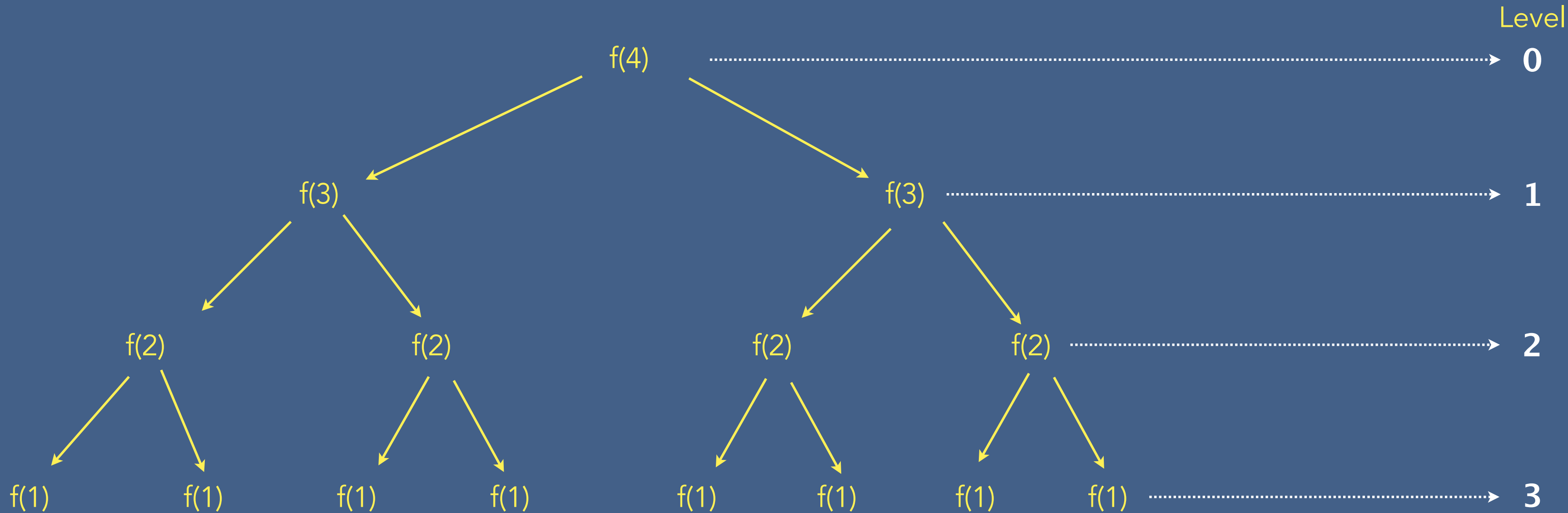
```
public int findMaxNumRec(int [] sampleArray, int n):  
    if (n == 1) {  
        return sampleArray[0]; }  
    return max(sampleArray[n-1], findMaxNumRec(sampleArray, n-1));
```

```
public int f(int n):  
    if (n <= 1) {  
        return 1; }  
    return f(n-1) + f(n-1);
```



How to measure Recursive Algorithm with multiple calls?

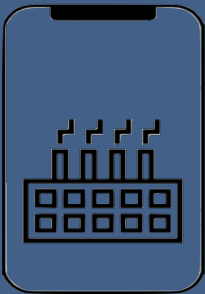
```
public int f(int n):  
    if (n <= 1) {  
        return 1; }  
    return f(n-1) + f(n-1);
```



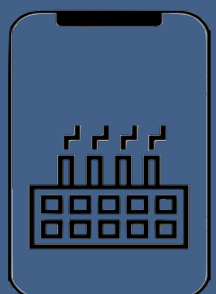
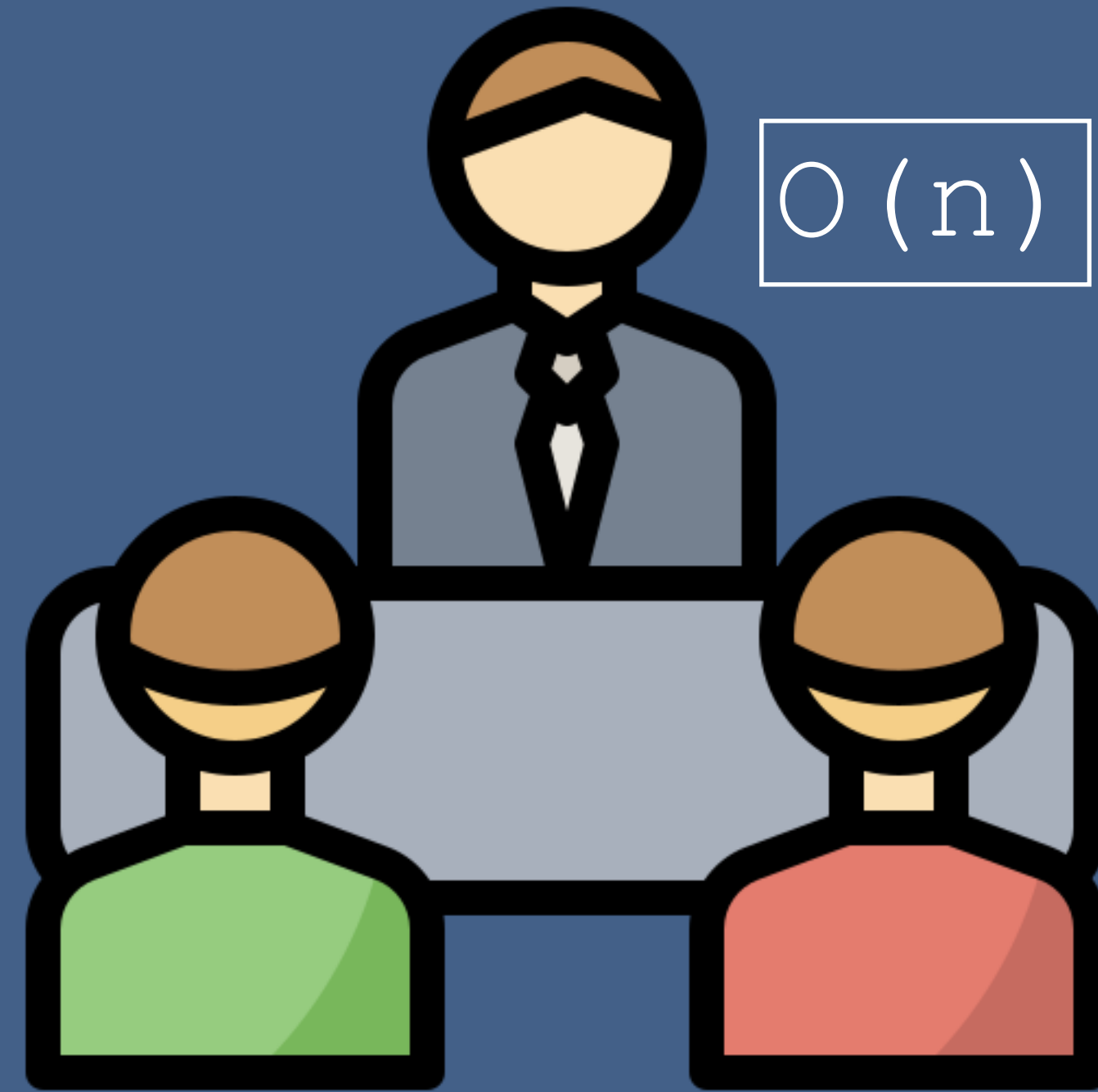
$2^0+2^1+2^2+...+2^n=2^{n+1}-1$
 $2^{n-1} \longrightarrow O(2^n)$

N	Level	Node#	Also can be expressed..	or..
4	0	1		2^0
3	1	2	$2 * \text{previous level} = 2$	2^1
2	2	4	$2 * \text{previous level} = 2 * 2^1=2^2$	2^2
1	3	8	$2 * \text{previous level} = 2 * 2^2=2^3$	2^3

$O(\text{branches}^{\text{depth}})$

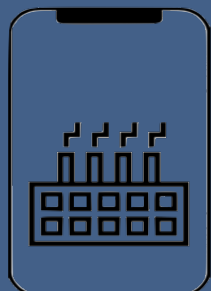


Big O Interview Questions



Interview Question 1

- Create a function which calculates the sum and product of elements of array
- Find the time complexity for created method.



Interview Question 2

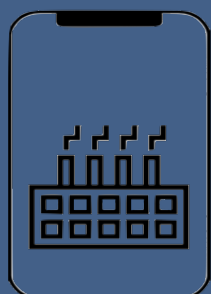
- Create a function which prints to the console the pairs from given array.
- Find the time complexity for created method.

[1,3,4,5] 11, 13, 14, 15

31, 33, 34, 35

41, 43, 44, 45

51, 53, 54, 55



Interview Question 3

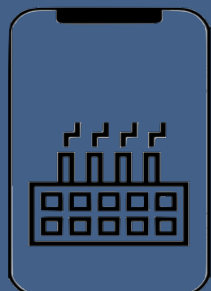
- What is the time complexity for this method?

```
void printUnorderedPairs(int[] array) {  
    for (int i=0; i<array.length; i++) {  
        for (int j=i+1; j<array.length; j++) {  
            System.out.println(array[i] + ", " + array[j]);  
        }  
    }  
}
```

[1,3,4,5] 13, 14, 15

34, 35

45



Interview Question 3

- What is the time complexity for this method?

```
void printUnorderedPairs(int[] array) {  
    for (int i=0; i<array.length; i++) {  
        for (int j=i+1; j<array.length; j++) {  
            System.out.println(array[i] + ", " + array[j]);  
        }  
    }  
}
```

1. Counting the iterations

1st → n-1
2nd → n-2
.
.
1

$(n-1) + (n-2) + (n-3) + \dots + 2 + 1$

$= 1 + 2 + \dots + (n-3) + (n-2) + (n-1)$

$= n(n-1)/2$

$= n^2/2 - n/2$

$= n^2$

2. Average Work

Outer loop - N times

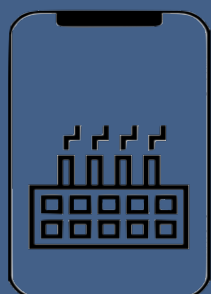
Inner loop?

1st → 10
2nd → 9
.
.
1

} = 5 → 10/2
n → n/2

$n * n/2 = n^2/2 \rightarrow O(N^2)$

Time Complexity : $O(N^2)$



Interview Question 4

What is the runtime of the below code?

```
void printUnorderedPairs (int[] arrayA, int[] arrayB) {  
    for (int i=0; i < arrayA.length; i++) {  
        for (int j=0; j < arrayB.length; j++) {  
            if (arrayA[i] < arrayB[j]) {  
                System.out.println(arrayA[i] + "," + arrayB[j]);  
            }  
        }  
    }  
}
```

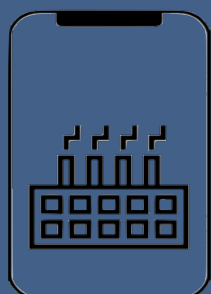
.....→ **O(ab)**

```
void printUnorderedPairs (int[] arrayA, int[] arrayB) {  
    for (int i=0; i < arrayA.length; i++) {  
        for (int j=0; j < arrayB.length; j++) {  
            O(1)  
        }  
    }  
}
```

a = arrayA.length;

b = arrayB.length;

Time Complexity : O(ab)



Interview Question 5

What is the runtime of the below code?

```
void printUnorderedPairs (int[] arrayA, int[] arrayB) {  
    for (int i=0; i < arrayA.length; i++) {  
        for (int j=0; j < arrayB.length; j++) {  
            for (int k = 0; k<1000000; k++) {  
                System.out.println(arrayA[i] + "," + arrayB[j]);  
            }  
        }  
    }  
}
```

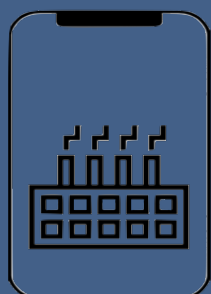
-----> **O(ab)**
-----> **O(1)**
-----> **O(1)**

b = arrayB.length;

a = arrayA.length;

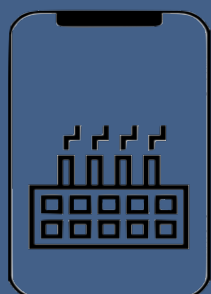
Time Complexity : O(ab)

100,000 units of work is still constant



Interview Question 6

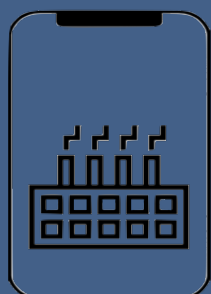
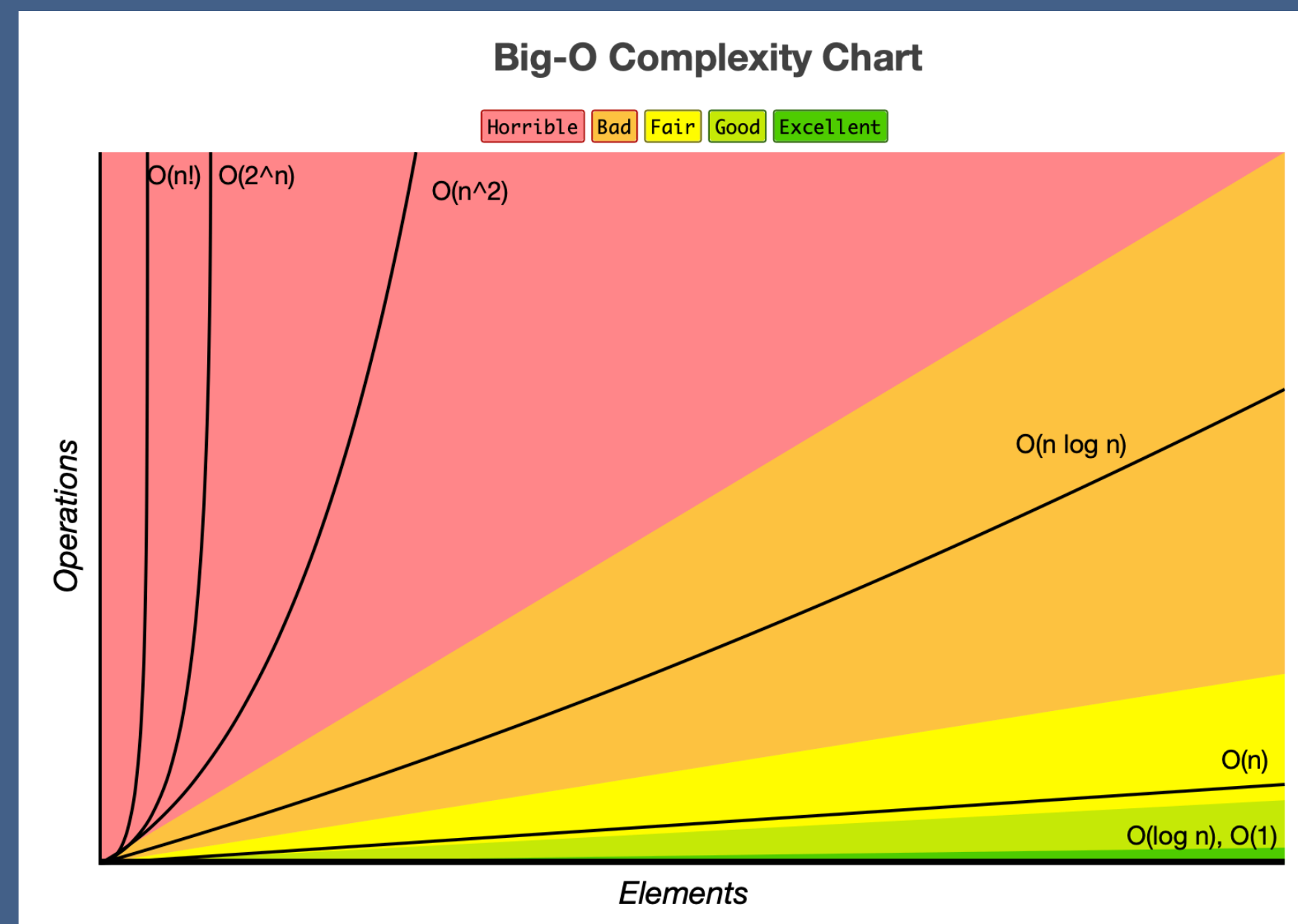
- Create a method which takes an array as a parameter and reverses it.
- Find the runtime of the created method?



Interview Question 7

Which of the following are equivalent to $O(N)$? Why?

1. $O(N + P)$, where $P < N/2$ $\longrightarrow O(N)$ ✓
2. $O(2N)$ $\longrightarrow O(N)$ ✓
3. $O(N + \log N)$ $\longrightarrow O(N)$ ✓
4. $O(N + N \log N)$ $\longrightarrow O(N \log N)$ ✗
5. $O(N + M)$ ✗



Interview Question 8

What is the runtime of the below code?

```
static int factorial(int n) { .....→ M(n)
    if (n < 0) {
        return -1;
    } else if (n == 0) { .....→ O(1)
        return 1;
    } else {
        return n * factorial(n-1); .....→ M(n-1)
    }
}
```

$$n! = 1*2*3*...*n$$

$$3! = 1*2*3=6$$

$$M(n)=O(1)+M(n-1)$$

$$M(1)=O(1)$$

$$M(n-1)=O(1)+M((n-1)-1)$$

$$M(n-2)=O(1)+M((n-2)-1)$$

}

$$M(n)=1+M(n-1)$$

$$=1+(1+M((n-1)-1))$$

$$=2+M(n-2)$$

$$=2+1+M((n-2)-1)$$

$$=3+M(n-3)$$

.

.

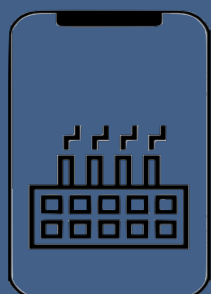
$$=a+M(n-a)$$

$$=n+M(n-n)$$

$$=n+1$$

$$=n$$

Time Complexity : O(N)



Interview Question 9

What is the runtime of the below code?

```
void allFib(int n) {  
    for (i=0; i<n; i++) {  
        fib(i);  
    }  
}  
  
static int fib(int n) {  
    if (n<=0) {  
        return 0;  
    } else if (n==1) {  
        return 1;  
    } else {  
        return fib(n-1) + fib(n-2);  
    }  
}
```

branches^{depth}→ $O(2^N)$

fib(1) → 2^1 steps

fib(2) → 2^2 steps

fib(3) → 2^3 steps

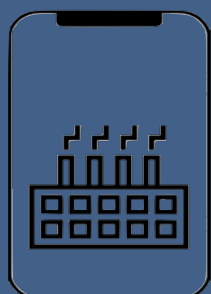
fib(4) → 2^4 steps

...

fib(n) → 2^n steps

→ Total work = $2^1 + 2^2 + 2^3 + 2^4 + \dots + 2^n$
 $= 2^{n+1} - 2$

Time Complexity : $O(2^N)$



Interview Question 10

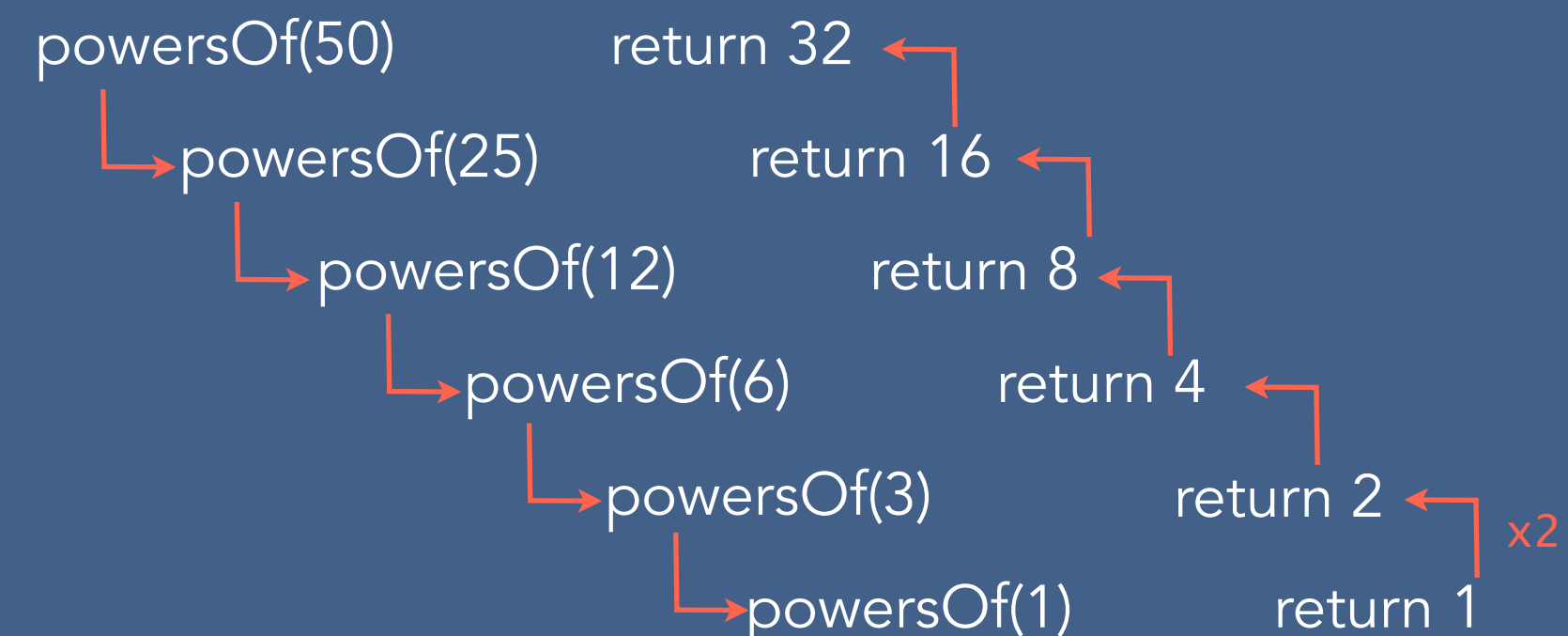
What is the runtime of the below code?

```
static int powersOf2(int n) {  
    if (n<1) {  
        return 0;  
    } else if (n==1) {  
        System.out.println(1);  
        return 1;  
    } else {  
        var prev = powersOf2(n/2);  
        var curr = prev*2;  
        System.out.println(curr);  
        return curr;  
    }  
}
```

.....→ **O(1)**

.....→ **O(1)**

n=50



Time Complexity : $O(\log N)$

